

Demo Script

System Purpose

The system is a platform for Pokémon trainers.

A user can register, log in, browse the full Pokémon catalog, build a personal Dream Team of up to 5 Pokémon, give them custom nicknames, and use AI-powered smart search and team analysis.

The backend is designed so that PokeAPI is consumed only by the server.

Pokémon data is stored locally in SQL Server to improve performance, availability, and advanced search capabilities even when PokeAPI is unavailable.

How to Run the Server

1. Open the project in Visual Studio.
2. Make sure SQL Server LocalDB is running.
3. Verify that the connection string exists in `appsettings.json`.
4. Run the database migrations:

Update-Database

1. Start the API project.
2. Verify that the server is running:

`GET /api/health`

Expected response:

```
{
  "status": "Healthy",
  "service": "Pokemon Trainer API"
}
```

Recommended Demo Flow

1. Register and Login

`POST /api/auth/register`

Then:

```
POST /api/auth/login
```

The login response returns a JWT token.
Use the token for all secured endpoints:

```
Authorization: Bearer {token}
```

2. Check Pokémon Import Status

```
GET /api/pokemon-import-status
```

If the response contains:

```
{  
  "isReady": true  
}
```

the system is ready to use.

Even if PokeAPI is unavailable, the system can continue working as long as a valid local Pokémon catalog exists in SQL Server.

3. Browse the Pokémon Catalog

```
GET /api/pokemon?page=1&pageSize=10
```

The data is returned from SQL Server, not directly from PokeAPI.

4. Smart Search

```
POST /api/pokemon-smart-search
```

Example body:

```
{
  "query": "find me top 10 small electric pokemon",
  "page": 1,
  "pageSize": 5
}
```

The system uses Gemini to understand the user's intent and convert the natural-language query into structured search criteria.

If Gemini is unavailable, the system falls back to a deterministic rule-based parser.

5. Build a Dream Team

```
POST /api/dream-team
```

Example body:

```
{
  "pokeApiId": 25,
  "nickname": "Sparky"
}
```

Each authenticated user can build a personal Dream Team of up to 5 Pokémon, without duplicates, and assign custom nicknames.

6. Analyze the Dream Team

```
GET /api/dream-team/analyze
```

The system calculates real metrics from the database:

- Pokémon types in the team
- Average HP, Attack, Defense, Speed, and other stats
- Fastest Pokémon
- Strongest Pokémon
- Best defensive Pokémon
- Strengths
- Weaknesses
- Recommendations
- Team score from 0 to 100

Gemini is used only to generate a friendly trainer-style summary.
The actual calculations are always performed on the server using SQL Server data.

7. Generate Nicknames

```
POST /api/pokemon/25/generate-nicknames
```

Body:

```
{
  "count": 5
}
```

The system returns nickname suggestions for the selected Pokémon.
If Gemini is available, the suggestions are AI-generated.
If Gemini is unavailable, the system uses deterministic fallback suggestions based on the Pokémon type.

Edge Case Handling

PokeAPI Is Unavailable

The system does not depend on PokeAPI in real time.
After a successful import, Pokémon data is stored in SQL Server, and the system can continue working even if PokeAPI is slow or unavailable.

Gemini Is Unavailable

Gemini is not the source of truth.
It is used only to improve the user experience.
If Gemini returns a timeout, 503, or an invalid response, the system falls back to internal deterministic logic.

Unexpected Errors

The API includes Global Error Handling Middleware that returns a consistent error response:

```
{
  "errorCode": "INTERNAL_SERVER_ERROR",
  "message": "An unexpected error occurred."
}
```

The full exception is logged on the server side with a TraceId.

Key Architecture Points

- Thin Controllers.
- Business Logic is implemented in dedicated services.
- SQL Server is the source of truth.
- PokeAPI is consumed only by the server.
- AI is used as an experience and interpretation layer, not as a data source.
- Fallback mechanisms exist for both PokeAPI and Gemini.
- Search-related indexes were added for common filtering and sorting fields.
- The backend is ready to be consumed by a future Angular Client.